

1) SVM constraint is: $y_i (w^T x_i + b) \geq 1$

Let's decode this constraint :-

2) We have 2 classes cat and dog :-

• Cat data points (say $y_i = +1$)

→ should lie on the positive side of the hyperplane

$$\rightarrow w^T x_i + b \geq +1$$

• Dog data points (say $y_i = -1$)

→ should lie on the negative side of the hyperplane

$$\rightarrow w^T x_i + b \leq -1$$

so the constraint is :-

$$\Rightarrow y_i (w^T x_i + b) \geq 1$$

It tells the optimiser where the each points should be placed relative to the decision boundary - cats on one side and dogs on other side.

2) How the constraint connects to the cost function

The cost function in SVM classification is:

$$\Rightarrow \text{Minimise } \frac{1}{2} \|w\|^2 + C \sum_i \epsilon_i$$

$$\Rightarrow y_i (w^T x_i + b) \geq 1 - \epsilon_i \quad (\text{this is our constraint}):$$

Here's the logic:

- The first part ($\frac{1}{2} \|w\|^2$) tells the optimiser:

- "Make the margin wide"

(smaller $\|w\|$ means larger margin)

- The constraint tells it:

- "But while doing that, don't move points across the wrong side of the hyperplane."

In other words, "keep cats on their side and dogs on theirs."

- The slack term (ϵ_i) lets the optimiser break the rule a little - but charges a penalty for each broken rule, controlled by C .

- In the previous one we wanted to minimize $\|w\|$ but also follow the constraint :-
- (Now - how do we make computers solve such problems?)
- We want to tell the computer:
 - "I want to minimize one thing, but I also have to respect another rule"
- The math trick to combine them is called the ~~Lagrangian~~ **Lagrangian** (pronounced "La-gran-jian")
- What is a Lagrangian?
 - Think of it as:
 - "A special formula that mixes your goal and your rule together, using a helper variable (called a **multiplier**) that keeps track of how strongly to enforce the rule"
- That helper variable is called the **Lagrange multiplier** - usually written as λ_i .

3) Building the Lagrangian for SVM step-by-step:

a) Cost function: $\frac{1}{2} \|w\|^2 + c \sum \epsilon_i$

Constraint: $y_i (w^T x_i + b) \geq 1 - \epsilon_i$

- Each data point must be on the correct side of the margin, but it can violate it a little (by ϵ_i) if necessary)

b) How optimizers like their constraints

- When we send this problem to a math optimizer (like a Quadratic Programming solver)
- It usually expects constraints in this form: $g(x) \leq 0$

That means: "This condition must be less than or equal to 0"

Eg if you have $x \geq 5$, you can rewrite it as:
 $5 - x \leq 0$ (They mean the exact same thing!)

So we just rearrange SVM's constraint to match that format

c) Rearrange it (pure algebra)

- Subtract $y_i (w^T x_i + b)$ from both sides: $0 \geq 1 - \epsilon_i - y_i (w^T x_i + b)$
- Flip it around: $1 - \epsilon_i - y_i (w^T x_i + b) \leq 0$

Now combine them: - add a term for each constraint to the cost function - multiplied by α_i (the Lagrange multiplier)

- $L = \frac{1}{2} \|w\|^2 + c \sum \epsilon_i + \sum \alpha_i [1 - \epsilon_i - y_i (w^T x_i + b)]$

This is called **Lagrangian Function**

4. What does this new formula do?

- The first part $\frac{1}{2} \|w\|^2 + c \sum_i \epsilon_i$
(makes margin wide + small penalty)

- The second part:

$$+ \sum_i \alpha_i [1 - \epsilon_i - y_i (w^T x_i + b)]$$

(adds the constraint into the cost)

If any point breaks the rule (i.e.)

$$y_i (w^T x_i + b) < 1 - \epsilon_i$$

then $[1 - \epsilon_i - y_i (w^T x_i + b)]$ becomes positive

the term $\alpha_i [1 - \epsilon_i - y_i (w^T x_i + b)]$ adds a penalty to the cost

Note:- cost function + constraint $\rightarrow$ Lagrangian $\rightarrow$ optimization solver

(is done automatically under the hood by libraries like sklearn, libsvm or cvxopt)

$\rightarrow$ (finding best possible solution)